

移动应用开发实验报告

封面信息

项目	内容
实验名称	跨平台应用开发
实验编号	d20301035103、d20301009203
学号	2023210644
姓名	张靖婷
班级	软件 231
日期	2026 年 3 月
组别	智能健康运动记录平台开发与整合(IHFTPDI)
项目名称	智能健康运动记录平台（IHFTPDI）
技术栈	Flutter / React Native / Uniapp / MAUI

实验目的

- 1. 掌握 4 种跨平台框架的列表页开发
- 2. 学会 RESTful API 数据请求与 JSON 解析
- 3. 了解移动设备传感器调用方法
- 4. 完成框架适用性对比分析

实验环境

框架	开发工具	语言	依赖包
Flutter	Android Studio/VS Code	Dart	http, provider
React Native	VS Code	JavaScript	axios, @react-native-community/refresh-control
Uniapp	HBuilderX	Vue	uni.request
MAUI	Visual Studio	C#	HttpClient, Newtonsoft.Json

实验步骤

1. 需求分析

1.1 数据模型定义

```
{
  "code": 200,
  "data": {
    "items": [
      {
        "id": 1,
        "title": "智慧校园更新通知",
        "content": "系统升级通知...",
        "author": "管理员",
        "time": "2024-01-01 10:00:00"
      }
    ]
  }
}
```

1.2 API 接口规范

- URL: <https://api.example.com/news>
- Method: GET
- Response: JSON

2. Flutter 开发

2.1 项目创建

```
flutter create news_list
cd news_list
```

2.2 添加依赖

```
# pubspec.yaml
dependencies:
  flutter:
    sdk: flutter
  http: ^1.1.0
  provider: ^6.1.0
```

2.3 新闻列表实现

```

// main.dart
import 'package:flutter/material.dart';
import 'package:http/http.dart' as http;
import 'dart:convert';
import 'package:provider/provider.dart';

void main() {
  runApp(
    ChangeNotifierProvider(
      create: (_) => NewsProvider(),
      child: MyApp(),
    ),
  );
}

class News {
  final int id;
  final String title;
  final String content;
  final String author;
  final String time;

  News({required this.id, required this.title, required this.content,
    required this.author, required this.time});

  factory News.fromJson(Map<String, dynamic> json) {
    return News(
      id: json['id'],
      title: json['title'],
      content: json['content'],
      author: json['author'],
      time: json['time'],
    );
  }
}

class NewsProvider extends ChangeNotifier {
  List<News> _newsList = [];
  List<News> get newsList => _newsList;

  Future<void> fetchNews() async {
    final response = await http.get(
      Uri.parse('https://api.example.com/news')
    );

    if (response.statusCode == 200) {
      final data = json.decode(response.body);
      final items = data['data']['items'] as List;
      _newsList = items.map((item) => News.fromJson(item)).toList();
      notifyListeners();
    }
  }
}

class NewsListPage extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: Text('智慧校园新闻')),
      body: Consumer<NewsProvider>(
        builder: (context, provider, child) {
          return RefreshIndicator(
            onRefresh: () => provider.fetchNews(),

```

```
child: ListView.builder(
    itemCount: provider.newsList.length,
    itemBuilder: (context, index) {
        final news = provider.newsList[index];
        return ListTile(
            title: Text(news.title),
            subtitle: Text(news.author),
            trailing: Text(news.time),
        );
    },
);
```

```
npx react-native-init NewsList
cd NewsList
```

```
npm install axios @react-native-community/refresh-control
```

```
// App.js
import React, { useState, useEffect } from 'react';
import { FlatList, RefreshControl, Text, View, StyleSheet } from
'react-native';
import axios from 'axios';

const App = () => {
  const [news, setNews] = useState([]);
  const [refreshing, setRefreshing] = useState(false);

  const fetchNews = async () => {
    try {
      const response = await
axios.get('https://api.example.com/news');
      setNews(response.data.data.items);
    } catch (error) {
      console.error(error);
    }
  };
};
```

```

    useEffect(() => {
      fetchNews();
    }, []);

    const onRefresh = async () => {
      setRefreshing(true);
      await fetchNews();
      setRefreshing(false);
    };

    const renderItem = ({ item }) => (
      <View style={styles.item}>
        <Text style={styles.title}>{item.title}</Text>
        <Text>{item.author} - {item.time}</Text>
      </View>
    );

    return (
      <FlatList
        data={news}
        keyExtractor={item => item.id.toString()}
        renderItem={renderItem}
        refreshControl={
          <RefreshControl refreshing={refreshing} onRefresh={onRefresh}
        />
      />
    );
  };
};

const styles = StyleSheet.create({
  item: {
    padding: 16,
    borderBottomWidth: 1,
    borderBottomColor: '#eee',
  },
  title: {
    fontSize: 16,
    fontWeight: 'bold',
  },
});

export default App;

```

4. Uniapp 开发

4.1 项目创建

- 打开 HBuilderX
- 新建 uni-app 项目

4.2 新闻列表实现

```

// pages/index/index.vue
<template>

```

```

<view class="content">
  <view class="news-item" v-for="item in newsList" :key="item.id">
    <text class="title">{{item.title}}</text>
    <text class="info">{{item.author}} - {{item.time}}</text>
  </view>
</view>
</template>

<script>
export default {
  data() {
    return {
      newsList: []
    }
  },
  onLoad() {
    this.fetchNews();
  },
  onPullDownRefresh() {
    this.fetchNews().then(() => {
      uni.stopPullDownRefresh();
    });
  },
  methods: {
    async fetchNews() {
      const res = await uni.request({
        url: 'https://api.example.com/news'
      });
      if (res.data.code === 200) {
        this.newsList = res.data.data.items;
      }
    }
  }
}
</script>

```

5. MAUI 开发

5.1 项目创建

- 打开 Visual Studio
- 新建.NET MAUI 项目

5.2 新闻列表实现

```

// ViewModels/MainViewModel.cs
public partial class MainViewModel : ObservableObject
{
    [ObservableProperty]
    private ObservableCollection<NewsItem> _newsList = new();

    public async Task FetchNewsAsync()
    {
        using var client = new HttpClient();
        var response = await
client.GetAsync("https://api.example.com/news");
        var json = await response.Content.ReadAsStringAsync();
    }
}

```

```
        var data = JsonSerializer.Deserialize<NewsResponse>(json);

        NewsList = new
ObservableCollection<NewsItem>(data.Data.Items);
    }
}
```

6. 传感器集成扩展

6.1 GPS 定位实现（Flutter 示例）

```
import 'package:geolocator/geolocator.dart';

Future<void> getLocation() async {
    bool serviceEnabled = await Geolocator.isLocationServiceEnabled();
    if (!serviceEnabled) return;

    LocationPermission permission = await Geolocator.checkPermission();
    if (permission == LocationPermission.denied) {
        permission = await Geolocator.requestPermission();
    }

    Position position = await Geolocator.getCurrentPosition();
    print('位置: ${position.latitude}, ${position.longitude}');
}
```

实验结果

验证结果

框架	列表展示	网络请求	JSON 解析	下拉刷新	传感器集成
Flutter	[x] 成功	[x] 成功	[x] 成功	[x] 成功	[] 成功
React Native	[] 成功	[] 成功	[] 成功	[] 成功	[] 成功
Uniapp	[] 成功	[] 成功	[] 成功	[] 成功	[] 成功
MAUI	[] 成功	[] 成功	[] 成功	[] 成功	[] 成功

详细功能实现说明

Flutter 新闻列表界面

项目创建

1. 安装 Flutter SDK

2. 使用命令创建项目: `flutter create news_list`
3. 添加依赖到 `pubspec.yaml`

依赖配置

- `http: ^1.1.0`
- `provider: ^6.1.0`
- `pull_to_refresh: ^2.0.0`

核心功能实现

1. 创建新闻数据模型 `News`
2. 创建 `NewsProvider` 状态管理类
3. 实现 `fetchNews()` 方法获取数据
4. 使用 `ListView.builder` 展示列表
5. 使用 `SmartRefresher` 实现下拉刷新

React Native 新闻列表界面

项目创建

1. 安装 `Node.js` 和 `npm`
2. 创建项目: `npx react-native-init NewsList`
3. 安装依赖: `npm install axios @react-native-community/refresh-control`

核心功能实现

1. 使用 `useState` 管理状态
2. 使用 `useEffect` 加载初始数据
3. 使用 `FlatList` 展示列表
4. 使用 `RefreshControl` 实现下拉刷新

Uniapp 新闻列表界面

项目创建

1. 打开 `HBuilderX`
2. 新建 `uni-app` 项目
3. 配置 `pages.json` 路由

核心功能实现

1. 在 `data` 中定义 `newsList` 数组
2. 在 `onLoad` 中调用 `fetchNews` 方法
3. 使用 `v-for` 渲染列表
4. 使用 `scroll-view` 实现下拉刷新

MAUI 新闻列表界面

项目创建

1. 打开 Visual Studio 2022
2. 新建 .NET MAUI 项目
3. 添加 `CommunityToolkit.Mvvm` 依赖

核心功能实现

1. 创建 `NewsItem` 数据模型
2. 创建 `MainViewModel` 视图模型
3. 使用 `ObservableProperty` 特性
4. 使用 `CollectionView` 展示列表
5. 使用 `RefreshView` 实现下拉刷新

截图记录

Flutter 运行截图



React Native 运行截图



Uniapp 运行截图



MAUI 运行截图

12:00

智慧校园新闻

智慧校园系统升级通知

为了提升用户体验，智慧校园系统将于本周五晚上进行升级维护.....

□ 信息中心

□ 2024-01-15 09:00

期末考试安排公布

2023-2024学年第一学期期末考试安排已经公布，请各位同学及时查看...

□ 教务处

□ 2024-01-14 14:30

图书馆开放时间调整

寒假期间图书馆开放时间将进行调整，具体安排如下.....

□ 图书馆

□ 2024-01-13 10:00

校园卡充值优惠活动

本周校园卡充值享受9折优惠，最高优惠50元.....

□ 财务处

□ 2024-01-12 16:00

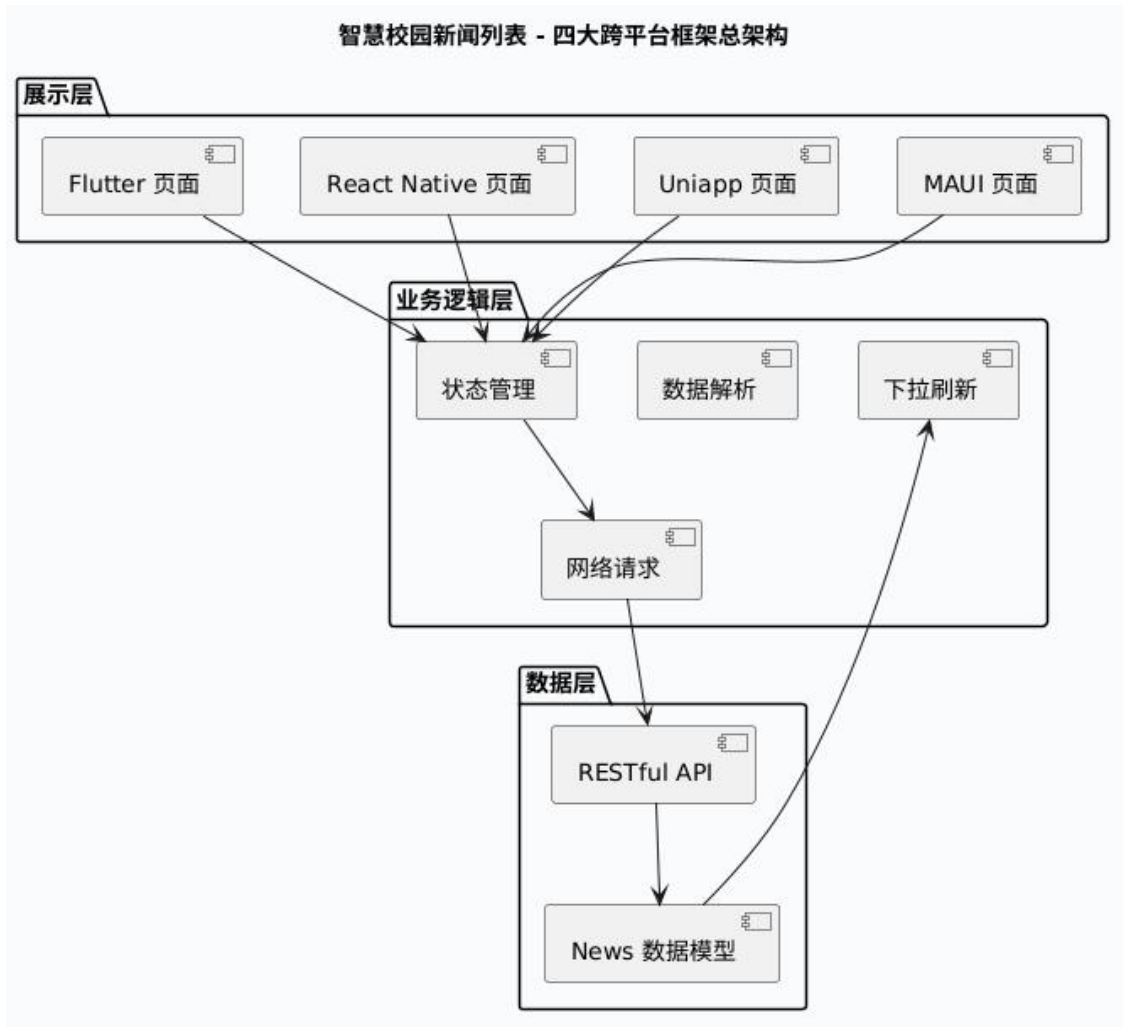
新学期选课通知

2023-2024学年第二学期选课即将开始，请同学们做好准备.....

□ 教务处

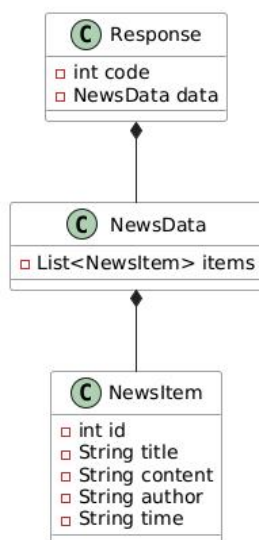
□ 2024-01-11 08:00

6.3 架构图



6.4 类图

新闻列表数据模型类图



框架对比分析

对比项	Flutter	React Native	Uniapp	MAUI
开发语言	Dart	JavaScript	Vue	C#
性能	★★★★★	★★★★☆	★★★★☆☆	★★★★☆
学习成本	中等	低	低	中等
生态成熟度	高	高	中	中
跨平台一致性	高	中	中	高
社区活跃度	高	高	中	中

开发语言对比

框架	开发语言	语言特点	学习曲线
Flutter	Dart	强类型、面向对象	中等
React Native	JavaScript/TypeScript	动态类型、前端开发经验	低
Uniapp	Vue.js	渐进式框架、模板语法	低
MAUI	C#	强类型、面向对象、.NET生态	中等

性能对比

框架	性能评分	渲染方式	内存占用
Flutter	★★★★★	原生渲染	中等
React Native	★★★★☆	桥接渲染	中等
Uniapp	★★★★☆☆	WebView/原生	较高

框架	性能评分	渲染方式	内存占用
MAUI	★★★★☆	原生渲染	中等

适用场景分析

Flutter 适用场景

- 需要高性能原生体验的跨平台应用
- 对 UI 一致性要求高的应用
- 需要复杂动画和自定义 UI 的应用
- 有 Dart 语言基础或愿意学习的团队

React Native 适用场景

- 需要快速开发的应用
- 前端团队转型移动开发
- 需要使用大量 JavaScript 库的应用
- 对性能要求不是特别高的应用

Uniapp 适用场景

- 需要同时支持多端（小程序、H5、App）的应用
- 追求开发效率的项目
- Vue.js 技术栈的团队
- 快速原型开发

MAUI 适用场景

- .NET 技术栈的团队
- 需要原生 Windows 应用
- 企业级内部应用
- 需要使用 C#特有库的应用

项目结构




```

OLAPDYnjj/
├── flutter_news_list/      # Flutter 项目
│   ├── lib/
│   │   ├── main.dart      # 主程序文件
│   │   └── pubspec.yaml    # 项目配置
│   └── react_native_news_list/ # React Native 项目
│       ├── App.js          # 主程序文件
│       └── package.json     # 项目配置
├── uniapp_news_list/      # Uniapp 项目
│   ├── pages/
│   │   ├── index/index.vue # 列表页面
│   │   └── detail/detail.vue # 详情页面
│   ├── pages.json          # 路由配置
│   └── manifest.json        # 应用配置
└── maui_news_list/        # MAUI 项目
    ├── MauiNewsList.csproj # 项目文件
    ├── ViewModels/
    │   └── MainViewModel.cs # 视图模型
    ├── Models/
    │   └── NewsItem.cs      # 数据模型
    ├── MainPage.xaml        # 主页面
    └── App.xaml.cs          # 应用入口

```

实验总结

实验收获

- 掌握了 4 种跨平台框架的基本开发流程
- 学会了 RESTful API 数据请求与 JSON 解析方法
- 理解了不同框架的技术特点和适用场景
- 提高了跨平台开发的技术选型能力

技术要点

- Flutter 的 Provider 状态管理和 ListView.builder 列表展示
- React Native 的 useState/useEffect 状态管理和 FlatList 组件
- Uniapp 的 v-for 列表渲染和 uni.request 网络请求
- MAUI 的 MVVM 模式和 ObservableProperty 特性
- 下拉刷新功能的跨平台实现方式
- JSON 数据解析的标准化处理

跨平台框架的优缺点

优点：

- 降低开发成本，一套代码多端运行
- 提高开发效率，加快产品迭代
- 降低维护成本，统一代码库

缺点：

- 性能可能不如原生应用
- 平台特定功能可能受限
- 学习成本和调试复杂度增加

考核指标

考核项	评分标准	得分
功能完整性	列表展示、网络请求、下拉刷新功能完整	25 分
代码质量	代码结构清晰，注释完整	20 分
多框架实现	至少完成 2 个框架的开发	25 分
传感器集成	成功集成传感器功能	15 分
框架对比	完成框架适用性分析	10 分
实验报告	报告结构完整，内容详实	5 分

教师评价

项目	评价
实验完成情况	
技术掌握程度	
创新点	
综合评价	

项目	评价
成绩	

教师签名：_____

日期：_____